WORKSHOP EPSI BAC+5 · DOSSIER D'INGÉNIERIE & DE GOUVERNANCE

RESILIENCE GUARD

Module de sécurité & de résilience « plug-and-protect » pour toute application

Le produit : une brique réutilisable qui se pose **devant n'importe quelle application**, sans toucher à son code, et lui ajoute automatiquement : WAF, KPI/observabilité, détection d'anomalies par IA, remédiation autonome (zéro-touch), haute disponibilité, et un assistant IA (RAG) — le tout piloté depuis un cockpit unique.

Démontré sur une app de paiement et une app vierge. Dépôt : github.com/bigwihodari/Resilience-Alpha (dossiers `module/` et `demo-apps/`). Environnement et données entièrement fictifs.

Consortium : Workshop2026-M2-G<n> · Octobre 2026
Équipe : Téo Debay · Timéo Quehen · Hodari Bigwi · Théo Vambre · Adrien Planquelle

1. Résumé exécutif

Problème. Protéger et superviser une application (WAF, métriques, détection d'attaque, reprise après incident, haute disponibilité) demande d'habitude d'intégrer et d'exploiter une dizaine d'outils, au cas par cas, dans chaque projet. C'est long, coûteux et rarement homogène.

Notre réponse : Resilience Guard. Un **module unique, réutilisable et autonome** qu'on installe **en une commande devant l'application à protéger**. Il apporte d'un bloc la défense (WAF), l'observabilité (KPI live), la détection par machine learning, la remédiation sans intervention humaine, la bascule haute disponibilité, et un assistant IA qui sait répondre sur ce qui se passe réellement sur le serveur (il lit les logs/erreurs en direct).

Valeur. On protège et supervise **n'importe quelle app** (dockerisée ou non) sans modifier son code, avec un cockpit prêt à l'emploi et une reprise automatique des incidents.

1 cmd installation devant l'app	**7** outils défensifs intégrés	**0** ligne de code à modifier dans l'app	**< 8 s** bascule sur le secours

NB : le dépôt contient aussi un démonstrateur « jeu » (bac à sable d'attaques) ; il n'est pas l'objet de la présentation — le livrable est le **module**.

2. Le module : architecture & installation

Principe « plug-and-protect »

L'application à protéger tourne **derrière la passerelle** du module (reverse-proxy). On renseigne une seule variable — l'URL amont de l'app — et on lance l'overlay Docker Compose :

```
PROTECTED_UPSTREAM=http://mon-app:8000 docker compose -f docker-compose.module.yml up -d
```

L'app est alors servie via la passerelle (port 8080), protégée et supervisée. Cockpit sur 8088, Grafana sur 3000. Pour une app non-dockerisée, `PROTECTED_UPSTREAM` pointe vers son URL.

Composants du module

Composant	Rôle	Pilier
passerelle (WAF + proxy)	Filtre les requêtes (signatures OWASP), proxifie vers l'app, expose les métriques	CYBER / DEV
Prometheus + Grafana + Loki	KPI (débit, latence, erreurs), tableaux de bord, logs centralisés	INFRA
détecteur (Isolation Forest)	Score d'anomalie sur le trafic observé par la passerelle	IA / DATA
orchestrateur	Remédiation autonome : bouclier, vérification, intégrité	CYBER / IA
assistant (RAG, Ollama local)	Répond aux questions d'exploitation ; lit les logs/erreurs en direct	IA / DATA
cockpit	Tableau de bord unique : santé, KPI, détection, remédiation, HA, menaces	DEV / TRANSFO
overlay HA	Reverse-proxy de bascule + passerelle de secours (failover/retour auto)	INFRA

La passerelle expose **exactement les mêmes métriques** que l'app de démo (`http_requests_total`, `http_request_duration_seconds`, `waf_blocked_total`) : détecteur, Prometheus et cockpit fonctionnent donc sans adaptation, quelle que soit l'app protégée.

Applications de démonstration

- **demo-apps/paiement** — une app métier (connexion + paiement) protégée par le module.
- **demo-apps/vitrine** — une app **vierge** (site statique) : on y pose le module tel quel pour montrer que « ça marche sur n'importe quoi ».

3. DEV & INFRA

Conteneurisation & overlay

Tout est conteneurisé (Docker Compose). Le module est un **overlay** (`docker-compose.module.yml`) qu'on superpose à l'app : aucune modification du code applicatif, découplage total. Les images de base sont pré-tirées pour des builds reproductibles hors-ligne (labo isolé).

Passerelle (reverse-proxy applicatif)

Écrite en FastAPI + httpx : pour chaque requête elle applique le WAF, gère le bouclier de délestage, proxifie vers l'app, mesure latence et statut, puis expose `/metrics` . Aucune dépendance à l'app protégée.

Observabilité

Prometheus agrège les métriques (débit, latence p95, erreurs 4xx/5xx, CPU/mémoire, requêtes bloquées). Grafana fournit les tableaux de bord ; Loki + Promtail centralisent les logs de tous les conteneurs. Le cockpit consomme ces sources pour une vue unique et temps réel.

4. IA / DATA

Détection d'anomalies (machine learning)

Un modèle **Isolation Forest** apprend le comportement « normal » du trafic (débit, erreurs) observé par la passerelle, puis calcule un **score d'anomalie** en continu. L'écart à la normale (distance standardisée) déclenche un drapeau au-delà d'un seuil, avec un plancher de bruit pour éviter les faux positifs au repos. Le modèle est persistant et ré-entraînable.

Détection par IP/acteur (probabiliste)

La suspicion est suivie **par IP** : elle monte avec les signaux (motifs d'attaque, pics WAF, retraits anormaux) et décroît avec le temps. Le blocage est **probabiliste et isolé** : plus la suspicion est haute, plus la probabilité de blocage augmente, et on ne bloque que l'IP suspecte — jamais tout le monde.

Assistant IA (RAG) auto-rempli

Un assistant local (Ollama, modèle ≤ 8B + embeddings) répond aux questions d'exploitation en s'appuyant sur la documentation (FAISS) **et sur les logs/erreurs du serveur lus en direct via Loki** : il sait dire ce qui se passe réellement (erreurs, blocages WAF, bascules). Sans privilèges : il lit, il n'agit pas.

5. CYBER — défense & remédiation

WAF applicatif

La passerelle inspecte chaque entrée (chemin, requête, corps) avec des signatures type OWASP (injection SQL, XSS…) et **bloque en 403** toute requête malveillante, en incrémentant `waf_blocked_total` pour la supervision.

Remédiation autonome (zéro-touch)

L'orchestrateur surveille le détecteur. Sur anomalie confirmée, il **décide puis exécute** une remédiation (bouclier anti-DoS / délestage via la passerelle), **vérifie** le retour à un état utilisable, et tient une **timeline** d'incident — sans intervention humaine. Accès maîtrisés : l'orchestrateur n'a pas de god-mode, il agit via une API d'administration restreinte et tracée.

Haute disponibilité (failover & retour)

L'overlay HA place un reverse-proxy de bascule devant une passerelle **principale** et une passerelle de **secours**. Si le principal tombe, le trafic bascule automatiquement sur le secours ; quand le principal se rétablit, l'état est réconcilié et il reprend la main — bascule et retour automatiques, visibles dans le cockpit.

6. TRANSFO — un produit réutilisable

Resilience Guard transforme une somme d'outils en **une capacité packagée, déployable partout**. L'intérêt « transformation » :

- **Standardisation** : même socle de sécurité/observabilité sur toutes les apps de l'organisation, au lieu d'intégrations hétérogènes.
- **Time-to-protect** : d'une app non protégée à une app supervisée + résiliente en une commande, sans projet d'intégration.
- **Autonomie d'exploitation** : remédiation sans astreinte humaine + assistant IA qui explique les incidents en langage clair.
- **Réversibilité** : overlay découplé, on l'ajoute ou on le retire sans toucher à l'app.

7. Démonstration & résultats (vérifiés en local)

Scénario type : on lance une app de démo, on pose le module devant (une commande), puis :

Étape	Observé
Trafic normal	Cockpit : KPI live (débit, latence, erreurs) ; détecteur **CLEAR**
Requête d'injection SQL (ex. `' OR 1=1--`)	**WAF 403** + `waf_blocked_total` incrémenté
Montée en charge / attaque	Score d'anomalie ↑ → orchestrateur active le bouclier, vérifie, journalise
Panne du principal	**Bascule automatique** sur le secours (< 8 s), retour auto au rétablissement
Question à l'assistant	Réponse en français citant les logs/erreurs réels (WAF, 403, bascules)

Vérifications réalisées en local avec Docker (passerelle sert l'app, /metrics expose les compteurs, cockpit à jour, WAF bloque l'injection). Les builds front sont validés avant déploiement.

8. Gouvernance & conformité

Analyse de risques (EBIOS RM) — synthèse

Risque	Impact	Mesure (apportée par le module)
Déni de service	Indisponibilité	Détection ML + bouclier de délestage + bascule HA
Injection (SQLi/XSS)	Compromission des données	WAF applicatif + contrôle d'intégrité
Panne d'un nœud	Interruption de service	Failover/retour automatiques, observabilité
Exploitation discrète	Vol / fuite	Suspicion par IP + blocage probabiliste isolé

PCA / PRA

La haute disponibilité (secours + bascule/retour automatiques) et la remédiation zéro-touch constituent un socle de continuité : le service se maintient et se rétablit sans intervention, avec journalisation d'incident pour l'après-coup.

NIS2 / DORA

Le module adresse des exigences clés : détection et gestion des incidents, résilience opérationnelle (bascule, reprise), journalisation et supervision, mesures techniques de protection (WAF). Il industrialise ces contrôles de façon homogène et réutilisable.

Recommandations à la direction

- Adopter le module comme **socle de sécurité/observabilité standard** devant les apps.
- Brancher les alertes du cockpit sur l'astreinte ; conserver les logs centralisés (Loki).
- Rejouer régulièrement les scénarios d'incident (attaque → remédiation → bascule).

9. Contributions, limites & suites

Contribution de l'équipe

Membre	Axe principal
Téo Debay	Architecture du module, passerelle, intégration
Timéo Quehen	Observabilité (Prometheus/Grafana/Loki), cockpit
Hodari Bigwi	Détection IA, orchestrateur de remédiation
Théo Vambre	Haute disponibilité, bascule/retour, infra
Adrien Planquelle	Sécurité (WAF), assistant RAG, démonstrateurs

Limites assumées

- Démonstrateur pédagogique : données et attaques fictives, labo isolé.
- WAF à base de signatures (pas de WAF managé) ; modèle ML volontairement simple et explicable.
- HA applicative (réconciliation d'état simple), pas un cluster de production.

En contexte industriel

- Passerelle durcie (mTLS, rate-limiting fin), WAF enrichi, secrets gérés (Vault/SOPS).
- Détection enrichie (features multiples, apprentissage continu supervisé).
- HA orchestrée (Kubernetes) + stockage répliqué ; intégration SIEM/SOAR.

Projet pédagogique — dépôt github.com/bigwihodari/Resilience-Alpha · module : `module/README.md` · données et attaques fictives, limitées au labo.